Lesson 2.3 **Duration: 40 minutes**

List Comprehension

Learning Objectives

- Write list comprehensions to create lists concisely and efficiently
- Apply conditions and nested logic within list comprehensions
- Compare list comprehensions with traditional loops for readability and performance

Engage (5-7 minutes)

Writing a loop to create a new list takes four to five lines. List comprehensions condense this into a single elegant line—like writing a sentence instead of a paragraph. They are a signature feature of Python that makes code more readable and often faster. Today we master this powerful technique for concise data transformation.

Explore (10-12 minutes)

Convert this loop into a list comprehension: `squares = []` followed by `for x in range(10): squares.append(x**2)`. Now create a list comprehension that generates only even squares. Then try a nested comprehension: create a list of all pairs (i, j) where i and j range from 0 to 3. Compare the readability of your comprehension with the equivalent loop version.

Explain (10-12 minutes)

List comprehensions provide a concise way to create lists. The syntax is `[expression for item in iterable if condition]`. The expression transforms each item, the for loop iterates, and the if clause filters. You can nest loops: `[expression for i in iterable1 for j in iterable2]`. List comprehensions are generally faster than equivalent for loops because they are optimized internally. However, avoid making them too complex—nested comprehensions with multiple conditions can become hard to read.

Elaborate (8-10 minutes)

Write list comprehensions for: generating all prime numbers up to 100, creating a matrix transpose from a 3x4 matrix, flattening a 2D list into 1D, extracting all vowels from a sentence, and computing the dot product of two vectors. Then refactor each to use traditional loops and compare performance using time measurement. Discuss when comprehensions are appropriate and when loops are clearer.

Evaluate (5-8 minutes)

Convert these loops to list comprehensions: (1) filter positive numbers from a list, (2) convert a list of names to uppercase, (3) create a list of tuples pairing numbers with their squares. Then write one comprehension with a nested loop and explain what it produces.